

SQL Interview Guide

Junior | Mid-Level | Senior

100 Essential Questions with Answers & Code Examples

This guide is organized by experience level. Junior questions (Q1–Q35) cover fundamentals. Mid-level (Q36–Q70) cover intermediate SQL skills. Senior (Q71–Q100) cover advanced topics, optimization, and real-world scenarios.

■ JUNIOR LEVEL | Q1 – Q35 (Basics & Fundamentals)

Q1. What is SQL?

SQL (Structured Query Language) is the standard language for managing relational databases. It is used to store, retrieve, update, and delete data, and to define database structures.

```
SELECT * FROM Employees;
```

Q2. What is a Database?

An organized collection of data stored electronically, allowing users to efficiently store, retrieve, update, and manage data. Example: a customer database storing names, emails, and purchase history.

Q3. What are the types of SQL commands?

DDL (CREATE, ALTER, DROP, TRUNCATE) – defines structure. DML (SELECT, INSERT, UPDATE, DELETE) – manipulates data. DCL (GRANT, REVOKE) – controls access. TCL (COMMIT, ROLLBACK, SAVEPOINT) – manages transactions. DQL (SELECT) – queries data.

Q4. What is a PRIMARY KEY?

A column (or combination) that uniquely identifies each row. Cannot be NULL, must be unique, and only one is allowed per table. Automatically creates a clustered index.

```
CREATE TABLE Employees (  
  EmployeeID INT PRIMARY KEY,  
  Name VARCHAR(50),  
  Age INT  
);
```

Q5. What is a FOREIGN KEY?

A column referencing the PRIMARY KEY of another table. Enforces referential integrity. Can contain duplicate values and NULLs.

```
ALTER TABLE Orders
ADD CONSTRAINT FK_Cust
FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID);
```

Q6. What is a UNIQUE Key?

Ensures all values in a column are distinct. Allows one NULL (in most databases). Multiple UNIQUE keys can exist in one table. Creates a non-clustered index.

```
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    Email VARCHAR(100) UNIQUE
);
```

Q7. Difference: Primary Key vs UNIQUE Key?

PRIMARY KEY: no NULLs, only one per table, creates clustered index, identifies a record. UNIQUE KEY: allows one NULL, multiple per table, creates non-clustered index, enforces uniqueness.

Q8. What is NOT NULL constraint?

Ensures a column cannot store NULL values. Useful for mandatory fields like Name or Email.

```
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Email VARCHAR(100)
);
```

Q9. What is DEFAULT constraint?

Automatically inserts a predefined value when no value is provided during INSERT.

```
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    Salary DECIMAL(10,2) DEFAULT 5000
);
```

Q10. Difference: DELETE, TRUNCATE, DROP?

DELETE – removes specific rows (DML), can be rolled back, slow (row by row). TRUNCATE – removes all rows, resets identity, cannot be rolled back, fast. DROP – deletes entire table including structure, fastest.

```
DELETE FROM Employees WHERE EmployeeID = 101;
TRUNCATE TABLE Employees;
DROP TABLE Employees;
```

■ *Tip: Use DELETE for selective removal, TRUNCATE to clear a table, DROP to remove it entirely.*

Q11. Difference: WHERE vs HAVING?

WHERE filters individual rows BEFORE grouping. HAVING filters groups AFTER GROUP BY. WHERE cannot use aggregate functions; HAVING can.

```
SELECT Dept, AVG(Salary) AS Avg
FROM Employees
WHERE Salary > 3000
GROUP BY Dept
HAVING AVG(Salary) > 5000;
```

Q12. What are the types of JOINS?

INNER JOIN – matching rows only. LEFT JOIN – all left rows + matching right (NULL if no match). RIGHT JOIN – all right rows + matching left. FULL JOIN – all rows from both tables. SELF JOIN – table joined with

itself. CROSS JOIN – Cartesian product.

Q13. What is INNER JOIN?

Returns only rows with matching values in both tables. Most commonly used join.

```
SELECT e.Name, d.DepartmentName
FROM Employees e
INNER JOIN Departments d ON e.DeptID = d.DeptID;
```

Q14. What is LEFT JOIN?

Returns all rows from the left table and matching rows from the right. NULL is shown for non-matching right-side columns.

```
SELECT e.Name, d.DepartmentName
FROM Employees e
LEFT JOIN Departments d ON e.DeptID = d.DeptID;
```

Q15. What is RIGHT JOIN?

Returns all rows from the right table and matching rows from the left.

```
SELECT e.Name, d.DepartmentName
FROM Employees e
RIGHT JOIN Departments d ON e.DeptID = d.DeptID;
```

Q16. What is FULL JOIN?

Returns all rows from both tables. NULL where no match exists on either side.

```
SELECT e.Name, d.DepartmentName
FROM Employees e
FULL JOIN Departments d ON e.DeptID = d.DeptID;
```

Q17. What is SELF JOIN?

A table joined with itself. Used for hierarchical data like employee-manager relationships.

```
SELECT E1.Name AS Employee, E2.Name AS Manager
FROM Employees E1
JOIN Employees E2 ON E1.ManagerID = E2.EmployeeID;
```

Q18. What is CROSS JOIN?

Returns the Cartesian product — every row from table1 combined with every row from table2. Result rows = Rows(T1) x Rows(T2).

```
SELECT e.Name, d.DepartmentName
FROM Employees e
CROSS JOIN Departments d;
```

Q19. What is UNION vs UNION ALL?

UNION combines results and removes duplicates (slower, sorts result). UNION ALL combines results keeping all duplicates (faster, no sorting). Both queries must have the same number of columns with compatible types.

```
SELECT Name FROM Employees
UNION ALL
SELECT Name FROM Managers;
```

Q20. What is Normalization?

Organizing data to reduce redundancy and improve integrity by dividing tables and defining relationships. 1NF – atomic values, no duplicate columns. 2NF – no partial dependency. 3NF – no transitive dependency. BCNF – every determinant is a candidate key.

Q21. What is Denormalization?

Intentionally adding redundancy to improve read performance at the cost of storage and consistency. Used in data warehouses and reporting systems where fast reads matter more.

Q22. Difference: CHAR vs VARCHAR?

CHAR(n) – fixed length, always uses n bytes, faster for fixed-size data (e.g. country codes). VARCHAR(n) – variable length, uses only needed bytes + 2 bytes overhead, better for names/emails.

```
CREATE TABLE T (Name CHAR(10), Address VARCHAR(50));
```

Q23. What is Auto Increment / Identity?

Automatically generates a unique sequential number for each new row. Used for primary key columns. In SQL Server use IDENTITY(1,1); in MySQL use AUTO_INCREMENT.

```
-- SQL Server:
EmployeeID INT IDENTITY(1,1) PRIMARY KEY
-- MySQL:
EmployeeID INT AUTO_INCREMENT PRIMARY KEY
```

Q24. What is GROUP BY?

Groups rows with the same values into summary rows. Used with aggregate functions (COUNT, SUM, AVG, MAX, MIN). Comes before ORDER BY in query order.

```
SELECT Department, COUNT(*) AS HeadCount, AVG(Salary) AS AvgSal
FROM Employees
GROUP BY Department;
```

Q25. Difference: GROUP BY vs ORDER BY?

GROUP BY – groups rows for aggregation, produces summary rows. ORDER BY – sorts the final result set ASC or DESC. Comes after GROUP BY. Does not require aggregate functions.

Q26. What is LIMIT / TOP?

Restricts how many rows a query returns. MySQL: LIMIT n SQL Server: SELECT TOP n

```
-- MySQL:
SELECT * FROM Employees ORDER BY Salary DESC LIMIT 5;
-- SQL Server:
SELECT TOP 5 * FROM Employees ORDER BY Salary DESC;
```

Q27. What is NULL in SQL?

NULL represents a missing or unknown value. Use IS NULL / IS NOT NULL to check for it — never use = NULL.

```
SELECT * FROM Employees WHERE ManagerID IS NULL;
```

■ *Tip: NULL != NULL — two NULLs are never equal in SQL.*

Q28. What is COALESCE?

Returns the first non-NULL value from a list of expressions. Widely supported across all databases.

```
SELECT Name, COALESCE(Email, Phone, 'No Contact') AS ContactInfo
FROM Students;
```

Q29. What is the CASE statement?

Adds conditional logic (like IF-ELSE) inside a SQL query.

```
SELECT Name, Salary,
CASE
    WHEN Salary > 6000 THEN 'High'
    WHEN Salary BETWEEN 4000 AND 6000 THEN 'Medium'
    ELSE 'Low'
END AS SalaryCategory
FROM Employees;
```

Q30. What is a Subquery?

A SELECT statement nested inside another SQL statement. Executes before the outer query. Types: single-row, multi-row, correlated.

```
SELECT Name FROM Employees
WHERE Salary = (SELECT MAX(Salary) FROM Employees);
```

Q31. How to find duplicate records?

Use GROUP BY on the target column and filter with HAVING COUNT(*) > 1.

```
SELECT Email, COUNT(*) AS Cnt
FROM Employees
GROUP BY Email
HAVING COUNT(*) > 1;
```

Q32. How to find the 2nd highest salary?

Use a subquery with MAX, or use ROW_NUMBER() / DENSE_RANK() window functions.

```
-- Subquery:
SELECT MAX(Salary) FROM Employees
WHERE Salary < (SELECT MAX(Salary) FROM Employees);

-- SQL Server ROW_NUMBER:
SELECT Salary FROM (
    SELECT Salary, ROW_NUMBER() OVER (ORDER BY Salary DESC) rn
    FROM Employees) t WHERE rn = 2;
```

■ *Tip: Interviewers love this question — know both approaches!*

Q33. What is a View?

A virtual table based on a SELECT query. Does not store data physically. Simplifies complex queries, provides security, and auto-updates when base table changes.

```
CREATE VIEW HighEarners AS
SELECT Name, Salary FROM Employees WHERE Salary > 8000;
```

Q34. Difference: View vs Table?

View – virtual, no physical storage, based on SQL query, auto-updates. Table – physical, stores raw data, requires manual updates.

Q35. What is a Stored Procedure?

A saved group of SQL statements executed as a unit. Benefits: reusability, security, reduced network traffic, cached execution plan.

```
CREATE PROCEDURE GetEmpByDept @DeptID INT
AS BEGIN
    SELECT Name, Salary FROM Employees WHERE DeptID = @DeptID
```

```
END;
```

■ *Tip: Call with: EXEC GetEmpByDept @DeptID = 1*

■ MID-LEVEL | Q36 – Q70 (Intermediate Skills)

Q36. Difference: Function vs Stored Procedure?

Function – must return a value, usable in SELECT, input params only, cannot do DML. Stored Procedure – may or may not return, called with EXEC, supports input+output params, can do DML.

Q37. What is a Trigger?

Automatic SQL code executed when a specified DML event (INSERT/UPDATE/DELETE) occurs on a table. Cannot be called manually. Used for logging, validation, auditing.

```
CREATE TRIGGER trg_AfterInsert
AFTER INSERT ON Employees
FOR EACH ROW
BEGIN
    INSERT INTO Logs(msg) VALUES('New employee added');
END;
```

Q38. What is a Cursor?

A database object that processes a result set row-by-row. Slower than set-based operations. Use steps: DECLARE, OPEN, FETCH NEXT, process in WHILE loop, CLOSE, DEALLOCATE.

```
DECLARE emp_cursor CURSOR FOR SELECT Name FROM Employees;
OPEN emp_cursor;
FETCH NEXT FROM emp_cursor INTO @name;
WHILE @@FETCH_STATUS = 0
BEGIN
    PRINT @name;
    FETCH NEXT FROM emp_cursor INTO @name;
END;
CLOSE emp_cursor;
DEALLOCATE emp_cursor;
```

■ *Tip: Avoid cursors on large datasets — use set-based JOINS instead.*

Q39. What is a CTE (Common Table Expression)?

A named temporary result set defined with the WITH keyword. Lives only within one query. Improves readability and can reference itself (recursive CTE).

```
WITH HighEarnings AS (
    SELECT * FROM Employees WHERE Salary > 8000
)
SELECT Name, Salary FROM HighEarnings ORDER BY Salary DESC;
```

■ *Tip: Prefer CTEs over nested subqueries for readability.*

Q40. What is a Temporary Table?

A table that exists only during the session. Prefixed with # in SQL Server. Automatically deleted when session ends. Used for intermediate results.

```
-- SQL Server:
CREATE TABLE #TempEmp (ID INT, Name VARCHAR(50));

-- MySQL:
CREATE TEMPORARY TABLE temp_emp AS SELECT * FROM Employees WHERE Salary > 5000;
```

Q41. Difference: Temp Table vs CTE?

Temp Table – physical storage, reusable within session, supports indexes, slower for small data. CTE – logical only, exists for one query, no indexes, faster for small datasets.

Q42. What is a Correlated Subquery?

A subquery that references columns from the outer query. Re-executed for each outer row — can be slow.

```
-- Employees earning above their dept average:
SELECT Name, Salary FROM Employees e
WHERE Salary > (
    SELECT AVG(Salary) FROM Employees
    WHERE DeptID = e.DeptID
);
```

■ *Tip: For performance, consider replacing with a JOIN + GROUP BY.*

Q43. Difference: IN vs EXISTS?

IN – compares values against a list, works with static values or subqueries, slower with large data. EXISTS – checks if subquery returns any row, stops at first match, faster for large datasets.

```
-- EXISTS (preferred for large tables):
SELECT Name FROM Customers c
WHERE EXISTS (SELECT 1 FROM Orders o WHERE o.CustomerID = c.CustomerID);
```

Q44. What is ACID?

Atomicity – all operations succeed or all fail (no partial commit). Consistency – database stays valid before and after transaction. Isolation – concurrent transactions don't interfere. Durability – committed data survives system failures.

■ *Tip: Always be ready to explain ACID with a bank transfer example.*

Q45. What is a Transaction?

A unit of work where all SQL operations succeed together (COMMIT) or all are undone (ROLLBACK).

Commands: BEGIN TRANSACTION, COMMIT, ROLLBACK, SAVEPOINT.

```
BEGIN TRANSACTION;
    UPDATE Accounts SET Balance -= 500 WHERE ID = 1;
    UPDATE Accounts SET Balance += 500 WHERE ID = 2;
COMMIT;
```

Q46. Difference: COMMIT vs ROLLBACK?

COMMIT – permanently saves all changes in the transaction. Cannot be undone. ROLLBACK – undoes all changes, restoring the previous state. Used when an error occurs.

Q47. What is SAVEPOINT?

Sets a named point within a transaction to allow partial rollback without undoing everything.

```
BEGIN TRANSACTION;
    UPDATE Employees SET Salary = 5000 WHERE ID = 101;
    SAVEPOINT SP1;
    UPDATE Employees SET Salary = 7000 WHERE ID = 102;
    ROLLBACK TO SP1; -- only undo 2nd update
COMMIT;
```

Q48. What is Indexing?

A database object that speeds up data retrieval using a B-tree lookup structure. Types: Primary (auto on PK), Unique, Clustered, Non-Clustered.

```
CREATE INDEX IX_EmpEmail ON Employees(Email);
```


■ *Tip: Indexes speed up reads but slow down INSERT/UPDATE/DELETE.*

Q49. Clustered vs Non-Clustered Index?

Clustered – sorts and stores data rows physically in index order. Only ONE per table (usually PK).

Non-Clustered – separate structure storing pointers to data rows. Multiple allowed per table.

Q50. What is a Window Function?

Performs a calculation across a set of rows related to the current row without collapsing them. Uses the OVER() clause with optional PARTITION BY and ORDER BY.

```
SELECT Name, Salary,
       AVG(Salary) OVER (PARTITION BY DeptID) AS DeptAvg,
       ROW_NUMBER() OVER (ORDER BY Salary DESC) AS Rank
FROM Employees;
```

Q51. ROW_NUMBER vs RANK vs DENSE_RANK?

ROW_NUMBER() – unique sequential number, no ties (1,2,3,4). RANK() – same rank for ties, skips numbers (1,2,2,4). DENSE_RANK() – same rank for ties, no gaps (1,2,2,3).

```
SELECT Name, Salary,
       ROW_NUMBER() OVER (ORDER BY Salary DESC) AS RowNum,
       RANK() OVER (ORDER BY Salary DESC) AS Rnk,
       DENSE_RANK() OVER (ORDER BY Salary DESC) AS DenseRnk
FROM Employees;
```

Q52. What is LAG and LEAD?

LAG(col, n) – accesses a value from n rows BEFORE the current row. LEAD(col, n) – accesses a value from n rows AFTER the current row. Useful for comparing consecutive rows (e.g., month-over-month sales).

```
SELECT OrderDate, TotalAmount,
       LAG(TotalAmount) OVER (ORDER BY OrderDate) AS PrevAmount
FROM Orders;
```

Q53. How to find Nth highest salary?

Use DENSE_RANK() or OFFSET-FETCH. Always use DISTINCT to handle duplicate salaries.

```
-- MySQL (3rd highest):
SELECT DISTINCT Salary FROM Employees
ORDER BY Salary DESC LIMIT 1 OFFSET 2;

-- SQL Server:
SELECT Salary FROM (
    SELECT Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) rn
    FROM Employees) t WHERE rn = 3;
```

Q54. What is Normalization (detailed)?

1NF – atomic values, no repeating groups. 2NF – 1NF + no partial dependency (all non-key columns depend on entire PK). 3NF – 2NF + no transitive dependency (non-key columns don't depend on other non-key columns). BCNF – every determinant is a candidate key.

Q55. What is Index Fragmentation?

Occurs when index pages are not in sequential order, slowing queries. Internal fragmentation – unused space inside pages. External fragmentation – pages stored out of order. Fix with: ALTER INDEX ... REBUILD or REORGANIZE.

```
-- Check fragmentation (SQL Server):
SELECT * FROM sys.dm_db_index_physical_stats(DB_ID(), OBJECT_ID('Employees'), NULL, NULL, 'DETAILED');
```

Q56. What is a Recursive CTE?

A CTE that references itself to process hierarchical data (org charts, categories, bill of materials). Has an Anchor part (base case) and a Recursive part joined with UNION ALL.

```
WITH EmpCTE AS (
    SELECT EmpID, Name, ManagerID FROM Employees WHERE ManagerID IS NULL
    UNION ALL
    SELECT e.EmpID, e.Name, e.ManagerID
    FROM Employees e JOIN EmpCTE c ON e.ManagerID = c.EmpID
)
SELECT * FROM EmpCTE;
```

Q57. What is a Pivot Table in SQL?

Transforms row data into columns to create summary reports. SQL Server has a native PIVOT operator; MySQL uses CASE+SUM.

```
-- MySQL style:
SELECT Product,
    SUM(CASE WHEN Month='Jan' THEN Sales ELSE 0 END) AS Jan_Sales,
    SUM(CASE WHEN Month='Feb' THEN Sales ELSE 0 END) AS Feb_Sales
FROM Sales GROUP BY Product;
```

Q58. Difference: UNION vs JOIN?

UNION – combines result sets vertically (rows). Tables must have same columns/types. Removes duplicates. JOIN – combines tables horizontally (columns) based on a common key. No column-count restriction.

Q59. How to get the highest salary per department?

Use GROUP BY with MAX() aggregate function.

```
SELECT Department, MAX(Salary) AS HighestSalary
FROM Employees
GROUP BY Department;
```

Q60. How to delete duplicate records?

Use ROW_NUMBER() CTE to identify duplicates and delete rows where rn > 1.

```
WITH CTE AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY Email ORDER BY EmpID) rn
    FROM Employees
)
DELETE FROM Employees
WHERE EmpID IN (SELECT EmpID FROM CTE WHERE rn > 1);
```

■ *Tip: Always backup data before deleting duplicates!*

Q61. How to find employees earning more than their manager?

Use a Self JOIN to compare each employee's salary against their manager's.

```
SELECT e.Name AS Employee, e.Salary AS EmpSal,
    m.Name AS Manager, m.Salary AS MgrSal
FROM Employees e
JOIN Employees m ON e.ManagerID = m.EmployeeID
WHERE e.Salary > m.Salary;
```

Q62. How to handle NULL values?

IS NULL / IS NOT NULL – check for NULLs. COALESCE(col, default) – return first non-NULL. ISNULL(col, default) – SQL Server. IFNULL(col, default) – MySQL. NULLIF(a, b) – returns NULL if a = b.

```
SELECT Name, COALESCE(Salary, 0) AS Salary FROM Employees;
```

Q63. How to find employees without managers?

Filter where ManagerID IS NULL, or use a LEFT JOIN finding unmatched rows.

```
-- Simple:
SELECT Name FROM Employees WHERE ManagerID IS NULL;
-- LEFT JOIN approach:
SELECT e.Name FROM Employees e
LEFT JOIN Employees m ON e.ManagerID = m.EmployeeID
WHERE m.EmployeeID IS NULL;
```

Q64. What are string functions in SQL?

LEN/LENGTH – character count. LEFT/RIGHT – extract from start/end. SUBSTRING – extract substring. CHARINDEX/INSTR – find position. REPLACE – replace text. UPPER/LOWER – change case. TRIM – remove spaces. CONCAT – join strings.

```
SELECT LEFT(Name,3), UPPER(Email), REPLACE(Phone,'-','') FROM Employees;
```

Q65. What are date functions in SQL?

GETDATE()/NOW() – current datetime. DATEADD – add interval. DATEDIFF – difference. YEAR/MONTH/DAY – extract parts. FORMAT/CONVERT – format dates.

```
-- Age calculation:
SELECT Name, FLOOR(DATEDIFF(CURDATE(), DOB) / 365) AS Age FROM Employees;
-- Last 3 months:
SELECT * FROM Employees WHERE JoinDate >= DATEADD(MONTH, -3, GETDATE());
```

Q66. What are aggregate functions?

COUNT(*) – all rows. COUNT(col) – non-NULL only. SUM – total. AVG – average. MAX/MIN – highest/lowest. Used with GROUP BY.

```
SELECT DeptID, COUNT(*) AS n, AVG(Salary) AS avg, MAX(Salary) AS top
FROM Employees GROUP BY DeptID;
```

Q67. What is Dynamic SQL?

SQL statements constructed and executed at runtime as strings. Useful for flexible queries with varying conditions. Must use parameterized queries to prevent SQL injection.

```
-- SQL Server:
DECLARE @sql NVARCHAR(500)
SET @sql = 'SELECT * FROM Employees WHERE Dept = @d'
EXEC sp_executesql @sql, N'@d VARCHAR(50)', @d = 'IT';
```

■ *Tip: Always use sp_executesql, not EXEC(@sql), to prevent SQL injection.*

Q68. What is the difference between DROP, DELETE, TRUNCATE?

DROP – removes table and structure, no rollback, fastest. DELETE – removes specific rows with WHERE, can rollback, slow. TRUNCATE – removes all rows, resets identity, no rollback for structure, fast.

Q69. How to fetch alternate (odd/even) rows?

Use MOD() in MySQL or ROW_NUMBER() in SQL Server.

```
-- MySQL even rows:
SELECT * FROM Employees WHERE MOD(EmpID, 2) = 0;
-- SQL Server odd rows:
WITH R AS (SELECT *, ROW_NUMBER() OVER (ORDER BY EmpID) rn FROM Employees)
SELECT * FROM R WHERE rn % 2 = 1;
```

Q70. How to calculate a running total?

Use SUM() as a window function with ORDER BY inside OVER().

```
SELECT Name, Salary,
       SUM(Salary) OVER (ORDER BY EmpID) AS RunningTotal
FROM Employees;
```

■ SENIOR LEVEL | Q71 – Q100 (Advanced & Optimization)

Q71. What are transaction isolation levels?

READ UNCOMMITTED – dirty reads allowed. READ COMMITTED (default) – only reads committed data. REPEATABLE READ – rows read can't change during transaction. SERIALIZABLE – fully isolated, no phantom reads. SNAPSHOT – reads data as it was at transaction start.

■ *Tip: For .NET apps, READ COMMITTED SNAPSHOT is common in SQL Server.*

Q72. What is a deadlock and how to avoid it?

Deadlock: two transactions each hold a lock the other needs — both wait forever. SQL Server auto-detects and kills the 'victim'. Prevention: always access tables in same order, keep transactions short, use proper indexes, use READ COMMITTED instead of SERIALIZABLE.

```
-- Deadlock prevention: consistent table access order
BEGIN TRANSACTION;
    UPDATE Departments SET Budget = 500000 WHERE DeptID = 1;
    UPDATE Employees SET Salary = 60000 WHERE EmpID = 101;
COMMIT;
```

Q73. What is Query Optimization?

Improving SQL efficiency to reduce execution time and resource usage. Key techniques: use indexes, avoid SELECT *, use EXISTS over IN, avoid functions in WHERE, use INNER JOIN over OUTER JOIN, use LIMIT/TOP, analyze execution plans.

```
-- Avoid:
SELECT * FROM Employees WHERE UPPER(Name) = 'VINAY';
-- Better:
SELECT * FROM Employees WHERE Name = 'Vinay';
```

Q74. What is a Query Execution Plan?

A roadmap the database engine uses to execute a query. Shows: table scans vs index seeks, join methods, sort operations, estimated row counts. Use to identify bottlenecks.

```
-- MySQL:
EXPLAIN SELECT Name FROM Employees WHERE Dept = 'IT';
-- SQL Server:
SET SHOWPLAN_ALL ON;
SELECT Name FROM Employees WHERE Dept = 'IT';
SET SHOWPLAN_ALL OFF;
```

■ *Tip: Look for 'Table Scan' in execution plans — that usually means a missing index.*

Q75. What is a Covering Index?

An index that includes all columns needed by a query, so SQL can answer the query entirely from the index without accessing the base table (no key lookup — faster).

```
CREATE INDEX IX_Cover
ON Orders(CustomerID)
INCLUDE (OrderDate, TotalAmount);
```

Q76. What is Table Partitioning?

Divides a large table into smaller physical pieces while appearing as one table. Types: Range, List, Hash, Composite. Improves query performance and data management on large datasets.

```
-- MySQL Range:
CREATE TABLE Employees (
    EmpID INT, Name VARCHAR(50), JoinDate DATE
)
PARTITION BY RANGE (YEAR(JoinDate)) (
    PARTITION p1 VALUES LESS THAN (2022),
    PARTITION p2 VALUES LESS THAN (2023),
    PARTITION p3 VALUES LESS THAN (2024)
);
```

Q77. What is SQL injection and how to prevent it?

SQL injection: malicious SQL inserted through user input to read/modify/delete data. In .NET, always use parameterized queries or stored procedures — never concatenate user input.

```
// VULNERABLE:
string sql = "SELECT * FROM Users WHERE Name = '" + input + "'";
// SAFE:
var cmd = new SqlCommand("SELECT * FROM Users WHERE Name = @Name", conn);
cmd.Parameters.AddWithValue("@Name", input);
```

■ *Tip: This is one of the OWASP Top 10 vulnerabilities. Always mention it in .NET interviews.*

Q78. What is the N+1 query problem?

Running 1 query to fetch N records, then N more queries to fetch related data for each — very slow.
Solution: use JOIN or eager loading (Include() in EF) to get all data in one query.

```
// EF - BAD (lazy loading = N+1):
var orders = context.Orders.ToList();
foreach(var o in orders) { var c = o.Customer; } // 1 extra query per order!

// EF - GOOD:
var orders = context.Orders.Include(o => o.Customer).ToList();
```

Q79. Difference: OLTP vs OLAP?

OLTP – day-to-day transactions, normalized, fast INSERT/UPDATE/DELETE, many small queries. OLAP – analytics/reporting, denormalized, complex SELECT with aggregations, data warehouses. Example: banking system = OLTP; sales dashboard = OLAP.

Q80. What is connection pooling?

Reusing existing database connections rather than opening new ones for each request. ADO.NET uses it automatically. Dramatically improves performance under high load. Key settings: Min Pool Size, Max Pool Size, Connection Timeout.

Q81. How to find the department with highest employee count?

Use GROUP BY + COUNT + ORDER BY DESC + TOP 1.

```
SELECT TOP 1 Department, COUNT(EmpID) AS EmpCount
FROM Employees
GROUP BY Department
ORDER BY EmpCount DESC;
```

Q82. How to display duplicate records with their count?

Use GROUP BY + HAVING COUNT > 1.

```
SELECT Name, COUNT(*) AS DuplicateCount
FROM Employees
GROUP BY Name
```

```
HAVING COUNT(*) > 1;
```

Q83. How to find employees joined in the last 3 months?

Use DATEADD (SQL Server) or DATE_ADD (MySQL) with the WHERE clause.

```
-- SQL Server:
SELECT Name, JoinDate FROM Employees
WHERE JoinDate >= DATEADD(MONTH, -3, GETDATE());
-- MySQL:
SELECT Name, JoinDate FROM Employees
WHERE JoinDate >= DATE_ADD(CURDATE(), INTERVAL -3 MONTH);
```

Q84. How to find common records from two tables without JOIN?

Use IN with a subquery (MySQL) or INTERSECT operator (SQL Server/PostgreSQL).

```
-- Using IN:
SELECT Name FROM Employees WHERE Name IN (SELECT Name FROM Managers);
-- Using INTERSECT (SQL Server):
SELECT Name FROM Employees
INTERSECT
SELECT Name FROM Managers;
```

Q85. How to swap two column values?

In a single UPDATE statement, swap both columns simultaneously.

```
UPDATE Employees
SET FirstName = LastName,
    LastName = FirstName
WHERE EmpID = 101;

-- Or with temp variable (MySQL):
UPDATE Employees
SET FirstName = @tmp := FirstName,
    FirstName = LastName,
    LastName = @tmp;
```

Q86. How to find the highest salary without MAX()?

Use ORDER BY DESC + LIMIT 1, or use a subquery with ALL operator.

```
-- Method 1:
SELECT Salary FROM Employees ORDER BY Salary DESC LIMIT 1;
-- Method 2:
SELECT Salary FROM Employees E1
WHERE Salary >= ALL (SELECT Salary FROM Employees E2);
```

Q87. What are EXCEPT and INTERSECT operators?

EXCEPT – returns rows from the first query NOT in the second (like LEFT JOIN with NULL filter).

INTERSECT – returns rows that appear in BOTH queries (like INNER JOIN on all columns). Both available in SQL Server and PostgreSQL.

```
-- Employees who are NOT managers:
SELECT Name FROM Employees
EXCEPT
SELECT Name FROM Managers;
```

Q88. What is a User-Defined Function (UDF)?

A custom function stored in the database. Scalar UDF – returns a single value. Inline Table-Valued – returns a table (single SELECT). Multi-statement Table-Valued – returns a table (multiple statements).

```
CREATE FUNCTION dbo.GetSalaryGrade (@Salary DECIMAL)
RETURNS VARCHAR(10)
AS BEGIN
    RETURN CASE
        WHEN @Salary > 8000 THEN 'High'
        WHEN @Salary > 4000 THEN 'Medium'
        ELSE 'Low' END
END;
```

Q89. What is CROSS APPLY and OUTER APPLY?

CROSS APPLY – like INNER JOIN for table-valued functions; excludes rows where function returns empty. OUTER APPLY – like LEFT JOIN; includes rows even when function returns no results (NULL).

```
SELECT e.Name, t.TopSale
FROM Employees e
CROSS APPLY dbo.GetTopSale(e.EmpID) t;
```

Q90. What is the MERGE statement?

Combines INSERT, UPDATE, and DELETE in one atomic operation based on a source vs target match. Also called UPSERT.

```
MERGE Employees AS target
USING NewEmployees AS source
ON target.EmpID = source.EmpID
WHEN MATCHED THEN
    UPDATE SET target.Salary = source.Salary
WHEN NOT MATCHED THEN
    INSERT (EmpID, Name, Salary) VALUES (source.EmpID, source.Name, source.Salary)
WHEN NOT MATCHED BY SOURCE THEN
    DELETE;
```

Q91. What is GROUPING SETS?

Allows multiple GROUP BY aggregations in a single query — more efficient than UNION ALL of multiple queries.

```
SELECT Department, JobTitle, SUM(Salary)
FROM Employees
GROUP BY GROUPING SETS (
    (Department),
    (JobTitle),
    (Department, JobTitle),
    ()
);
```

Q92. What is ROLLUP and CUBE?

ROLLUP – generates subtotals and grand total for hierarchical groupings. CUBE – generates all possible combinations of groupings (more comprehensive than ROLLUP).

```
-- ROLLUP:
SELECT Dept, Year, SUM(Salary)
FROM Employees
GROUP BY ROLLUP(Dept, Year);
-- CUBE:
GROUP BY CUBE(Dept, Year);
```


Q93. What is the OFFSET-FETCH pattern?

Standard SQL Server paging syntax. OFFSET skips N rows; FETCH NEXT returns the next M rows. More standard than LIMIT/OFFSET in MySQL.

```
-- Page 3 with 10 rows per page:
SELECT Name, Salary FROM Employees
ORDER BY EmpID
OFFSET 20 ROWS
FETCH NEXT 10 ROWS ONLY;
```

Q94. How to handle errors in SQL Server?

Use TRY...CATCH blocks similar to C# exception handling. Use ERROR_MESSAGE(), ERROR_NUMBER() inside CATCH block.

```
BEGIN TRY
    BEGIN TRANSACTION;
        UPDATE Accounts SET Balance -= 500 WHERE ID = 1;
        UPDATE Accounts SET Balance += 500 WHERE ID = 2;
    COMMIT;
END TRY
BEGIN CATCH
    ROLLBACK;
    PRINT ERROR_MESSAGE();
END CATCH;
```

Q95. What are DDL Triggers?

Triggers that fire on DDL events (CREATE, ALTER, DROP) rather than DML. Used for auditing schema changes, preventing accidental drops, or enforcing naming conventions.

```
CREATE TRIGGER trg_PreventDrop
ON DATABASE
FOR DROP_TABLE
AS BEGIN
    ROLLBACK;
    PRINT 'Dropping tables is not allowed!';
END;
```

Q96. What is the difference between SNAPSHOT and READ COMMITTED SNAPSHOT?

SNAPSHOT isolation – each transaction sees data as it was at transaction start. READ COMMITTED SNAPSHOT – each statement sees latest committed data at statement start. Both avoid reader-writer blocking. RCSI is usually preferred for OLTP workloads.

Q97. What is sp_executesql and why prefer it over EXEC?

sp_executesql executes parameterized dynamic SQL. Benefits: prevents SQL injection, allows execution plan reuse (cached), supports output parameters.

```
EXEC sp_executesql
    N'SELECT * FROM Employees WHERE Dept = @Dept',
    N'@Dept VARCHAR(50)',
    @Dept = 'IT';
```

■ *Tip: Never concatenate user input into dynamic SQL — always parameterize.*

Q98. What are system catalog views in SQL Server?

Built-in views exposing metadata about the database. sys.tables – all tables. sys.columns – columns. sys.indexes – indexes. sys.dm_exec_query_stats – query performance stats.

sys.dm_db_index_physical_stats – index fragmentation.

```
-- List all tables:
SELECT name FROM sys.tables ORDER BY name;
-- Find all columns of a table:
SELECT name, system_type_id FROM sys.columns WHERE object_id = OBJECT_ID('Employees');
```

Q99. What is a Sequence object vs Identity?

IDENTITY – column property, auto-increments per row insert, tied to one table. SEQUENCE – independent database object, can be shared across multiple tables, generates numbers on demand with NEXT VALUE FOR.

```
CREATE SEQUENCE dbo.OrderSeq START WITH 1 INCREMENT BY 1;
SELECT NEXT VALUE FOR dbo.OrderSeq AS NextOrderID;
```

Q100. What is a GUID in SQL Server?

A Globally Unique Identifier — a 128-bit value virtually guaranteed to be unique worldwide. Use UNIQUEIDENTIFIER data type with NEWID() or NEWSEQUENTIALID(). Pros: unique across servers. Cons: large (16 bytes), causes index fragmentation if not sequential.

```
CREATE TABLE Orders (
    OrderID UNIQUEIDENTIFIER DEFAULT NEWSEQUENTIALID() PRIMARY KEY,
    CustomerID INT
);
```

■ *Tip: Use NEWSEQUENTIALID() instead of NEWID() as the default to reduce index fragmentation.*